

Arquitectura, operación y replicación

Una tienda en línea con control de ventas, sobre servicios gratuitos y sin servidor propio. Cómo está armada, por qué así, y cómo montarle una a otro cliente en menos de una hora.

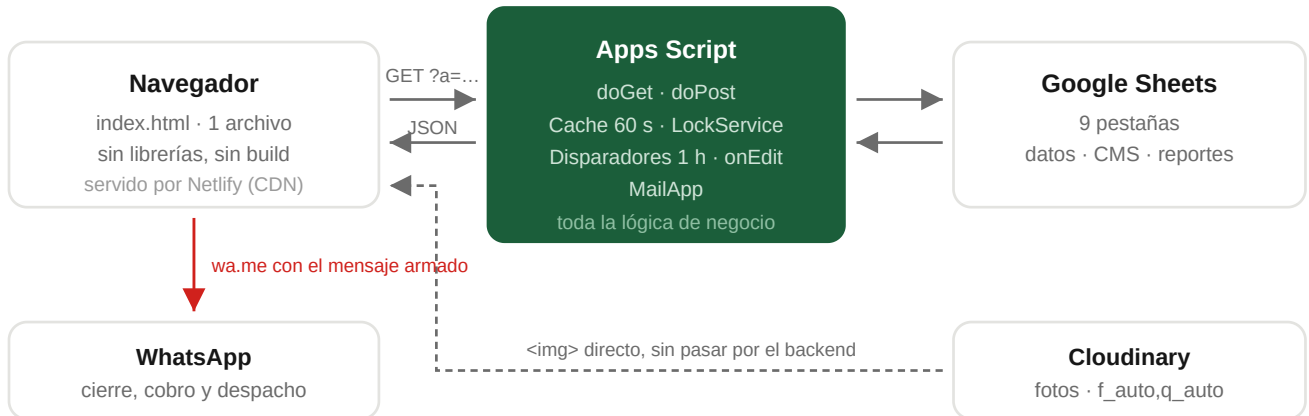
Septiembre de 2026 · Para quien la instala y la mantiene

CONTENIDO

- | | |
|--------------------------------|--------------------------------|
| 1. Arquitectura | 8. Capacidad y límites |
| 2. Las piezas | 9. Despliegue |
| 3. El contrato tienda–hoja | 10. Replicar para otro cliente |
| 4. Modelo de datos | 11. Pruebas |
| 5. Flujo de un pedido | 12. Decisiones y límites |
| 6. Idempotencia y concurrencia | 13. Mantenimiento |
| 7. Seguridad | |

1 Arquitectura

Un archivo HTML estático servido por CDN, y un Google Apps Script unido a una hoja de cálculo que hace de base de datos, CMS y backend. No hay servidor, no hay build, no hay dependencias.



El navegador habla con Apps Script por GET con JSON. Las fotos van directo del CDN de Cloudinary. El cierre de la venta pasa por WhatsApp, fuera del sistema.

LA PREMISA

La forma más costo-eficiente de poner en línea a un negocio pequeño, con control básico de ventas, usando servicios gratuitos. **Costo de operación: \$0/mes.** Único gasto opcional: el dominio, ~\$50.000/año. Cada decisión de este documento se sostiene contra esa vara.

2 Las piezas

ARCHIVO	QUÉ ES	DÓNDE VIVE
<code>index.html</code>	La tienda entera: HTML, CSS y JS en un archivo. ~100 KB.	Netlify
<code>_headers</code>	Cabeceras de seguridad de Netlify (CSP, <code>frame-ancestors</code> , etc.).	Netlify
<code>compartir.jpg</code>	Imagen 1200×630 de la vista previa al compartir.	Netlify
<code>apps-script.gs</code>	El backend completo. ~1.500 líneas, sin dependencias.	Apps Script, unido a la hoja

La carpeta `publicar/` del repositorio contiene **exactamente** esos tres primeros archivos y nada más. Existe para no arrastrar por accidente la documentación interna a un hosting público.

Reglas de construcción

- Un solo archivo. Sin frameworks, sin npm, sin paso de build.
- Sin `localStorage` ni `sessionStorage` .
- Mobile-first, probado a 375 px.
- Todo texto visible sale de la hoja o de una constante de respaldo en el archivo.

3 El contrato tienda–hoja

Apps Script se publica como aplicación web y expone cuatro operaciones por GET. Todas devuelven JSON con `version` .

PETICIÓN	DEVUELVE	CUÁNDO
?a=catalogo	Productos activos, envíos y configuración.	Al cargar la tienda.
?a=validar&items=...&cupon=...&envio=...&sub=...	Subtotal recalculado por la hoja, descuento, envío, total y avisos. Con <code>sellar=1</code> además deja la fila en <code>Validaciones</code> , con el número del pedido como referencia.	Al aplicar cupón y antes de enviar.
?a=registrar&pedido=...&items=...	Graba el pedido en <code>Pedidos</code> .	Al tocar «Enviar pedido».
?a=version	Solo la versión del contrato.	Diagnóstico.

El versionado, y el error que evita

`VERSION` en el script y `SCRIPT_VERSION` en la tienda deben coincidir. Si no coinciden, la tienda **no sella el pedido**, avisa por consola y el mensaje sale marcado como «calculado por la página», para que el error sea ruidoso en vez de silencioso.

LA CAUSA NÚMERO UNO DE FALLOS

En Apps Script, **guardar el código no publica nada**. La aplicación web sigue sirviendo la versión anterior hasta que se hace *Implementar* → *Gestionar implementaciones* → *lápiz* → *Versión: Nueva versión*. Casi todos los síntomas raros («la hoja no se llena», «el cupón no aplica») son esto. El chequeo de versión existe precisamente para delatarlo en un segundo.

`VERSION` describe el **contrato**, no el código. Un cambio que la tienda no consume —el tablero, el correo, el formato de una pestaña— **no** debe subirla: subirla obligaría al cliente a republicar Netlify sin razón.

Por qué GET y no POST

Apps Script responde a un POST con un **302 hacia** `script.googleusercontent.com`, y el navegador convierte esa redirección en un GET sin cuerpo. El resultado es una petición que parece exitosa y no escribe nada. Todo el tráfico va por GET por eso; los parámetros van en la URL y el servidor los valida.

4 Modelo de datos

Nueve pestañas. Los nombres son parte del contrato: renombrar una rompe el sistema en silencio. Agregar columnas **a la derecha** es seguro; insertarlas en medio no.

PESTAÑA	COLUMNAS	QUIÉN ESCRIBE
Configuración	Clave · Valor · Qué es	Dueño
Catálogo	ID · Nombre · Formato · Categoría · Precio · Stock · Descripción · Imágenes · Destacado · Activo	Dueño (Stock también el script)
Envíos	ID · Nombre · Valor	Dueño
Cupones	Código · Tipo · Valor · Mínimo · Vence · Usos máximos · Usos confirmados · Activo · Notas	Dueño (usos, el script)
Validaciones	Fecha · Pedido · Cupón · Subtotal hoja · Subtotal página · Discrepancia · Descuento · Envío · Total hoja · Detalle	Script
Pedidos	Fecha · Pedido · Validación · Estado · Ciudad · Cupón · Producto · ID · Cantidad · Precio · Subtotal línea · Total pedido · Inventario	Script (Estado, el dueño)
Tablero	4 columnas de presentación	Script, se reescribe entera
Más vendidos	Producto · ID · Unidades · Ingresos · Pedidos	Script
Errores	Fecha · Error · Primeros 200 caracteres	Script

Una fila por línea de pedido, no por pedido: el total del pedido se repite en cada fila. Todo cálculo agrupa por número de pedido antes de sumar totales — si no, un pedido de tres productos contaría triple.

Configuración: las claves

La pestaña `Configuración` es la que hace genérica la tienda. Cuando el script gana una clave nueva, volver a ejecutar `instalar()` la agrega a hojas ya existentes sin tocar los valores del dueño.

Identidad	<code>negocio</code> , <code>whatsapp</code> , <code>logo</code> , <code>favicon</code>
Portada	<code>portada_titulo</code> , <code>portada_texto</code> , <code>portada_puntos</code> , <code>como_compras</code> , <code>pie_descripcion</code>
Marca	<code>color_principal</code> , <code>color_secundario</code> , <code>color_alterno</code> — se pueden pintar en vez de escribir
Legales	<code>empresa_razon</code> , <code>empresa_nit</code> , <code>empresa_correo</code> , <code>empresa_direccion</code> , <code>empresa_ciudad</code> , <code>empresa_tel</code> , <code>legal_actualizado</code>
SEO	<code>sitio_url</code> , <code>sitio_titulo</code> , <code>sitio_descripcion</code>
Correo	<code>correo_resumen</code> , <code>correo_hora</code> , <code>correo_siempre</code> , <code>correo_ultimo</code>

Los valores entre corchetes — `[NIT]` , `[DIRECCIÓN]` — se tratan como «todavía no lo tengo» y la frase que los contiene no se muestra, en vez de renderizar el corchete.

5 Flujo de un pedido

- 1 Carga.**
 La tienda pide `?a=catalogo` , aplica configuración, catálogo, envíos y colores. Si no hay respuesta, usa el catálogo de respaldo del propio archivo y sigue funcionando.
- 2 Carrito.**
 Estado en memoria. `sanear()` lo recorta al stock, valida el envío y descarta un cupón inválido en cada repintado.
- 3 Validación.**
 Al aplicar cupón o al llenar el formulario, la tienda manda el carrito a `?a=validar&sellar=1` . La hoja recalcula con **sus** precios, aplica el cupón según **sus** reglas y devuelve el total con un código de verificación. La fila queda en `Validaciones` .
- 4 Sello.**
 El sello solo se considera vigente si coinciden versión, firma del carrito y subtotal. Cualquier cambio posterior lo invalida y obliga a revalidar.

5 Envío.

El mensaje lleva el número del pedido, no un código de verificación aparte: son el mismo identificador desde que se unificaron, y repetirlo solo ensuciaba el mensaje. El botón es un `<a href="wa.me/...">` real —no `window.open` — para no chocar con el bloqueo de ventanas emergentes ni generar dobles toques fantasma. Al hacer clic se dispara `?a=registrar` con `keepalive`.

6 Cierre.

El pedido entra como *Por confirmar*. El dueño confirma en la hoja; ahí baja el inventario y la venta entra en los reportes.

El mensaje de WhatsApp

Se recorta progresivamente para no pasar de 2.000 caracteres de URL: primero se caen las notas, luego el detalle por producto, luego las líneas de cortesía. **No lleva datos de pago** (ver Seguridad). Va por `encodeURIComponent`.

6 Idempotencia y concurrencia

Tres mecanismos, cada uno por un bug real de producción.

MECANISMO	PROBLEMA QUE RESUELVE
Número de pedido estable	Se calcula una vez por carrito y solo se reinicia cuando el carrito queda vacío. Antes se generaba en cada envío y un doble toque creaba dos pedidos.
Deduplicación en el servidor	<code>registrar</code> ignora un número de pedido ya grabado. Tres envíos del mismo pedido dejan una sola entrada.
<code>LockService</code> + <code>upsert</code> por número	<code>Validaciones</code> se escribe leyendo-y-escribiendo bajo candado. Sin él, dos validaciones simultáneas creaban dos filas con códigos distintos.
Columna <code>Inventario</code>	Marca cada línea como <i>Descontado</i> o <i>Devuelto</i> . Hace que confirmar, anular y volver a confirmar no descuadre el stock, y que la rutina se pueda correr mil veces sin efecto.
Congelado tras el envío	Una fila de <code>Validaciones</code> cuyo pedido ya se registró no se puede reescribir.

El inventario **no** se descuenta al enviar el pedido, a propósito: un pedido abierto en WhatsApp no es una venta, y si descontara ahí, cualquiera podría dejar el inventario en cero abriendo pedidos que nunca paga.

7 Seguridad

El modelo de amenaza parte de un hecho: **todo lo que está en el navegador es del atacante**. El HTML se lee, el JavaScript se edita, la consola está abierta.

RIESGO	MITIGACIÓN
Alterar el total desde la consola	El servidor recalcula todo con los precios de la hoja. La tienda nunca es la autoridad sobre el precio. Además <code>Object.freeze</code> sobre catálogo, cupones y envíos.
Inventar o reutilizar un cupón	Los cupones viven solo en la hoja, con vigencia, mínimo y tope de usos. La lista del archivo está vacía.
Clonar el sitio y cambiar la llave de pago	Los datos de pago no están en la página. Se entregan por respuesta automática de WhatsApp, que además advierte al cliente que no transfiera si ve otra llave. Eso hace inútil una copia.
Inyección de fórmulas en Sheets	<code>celdaSegura()</code> antepone un apóstrofo a todo valor que empiece por <code>=</code> , <code>+</code> , <code>-</code> , <code>@</code> o un carácter de control, y recorta a 60 caracteres.
XSS y carga de recursos ajenos	CSP en la etiqueta <code>meta</code> y en <code>_headers</code> : <code>default-src 'none'</code> , con lista explícita para estilos, tipografías, imágenes y <code>connect-src</code> . <code>frame-ancestors</code> solo funciona en cabecera, por eso existe <code>_headers</code> .
Payloads absurdos al backend	Tope de 30 ítems, cantidad máxima 200, total máximo 5.000.000, IDs que no estén en el catálogo se descartan, duplicados se colapsan y el <code>Estado</code> nunca lo decide quien envía.
Fuga de datos personales	La hoja guarda qué se pidió, no quién . Nombre, teléfono y dirección viven solo en WhatsApp. Lo que no se guarda no se filtra.

LO QUE ESTE DISEÑO NO PUEDE IMPEDIR

wa.me solo rellena la caja de texto: **el cliente puede editar el mensaje antes de enviarlo**. Por eso el mensaje es lo que el cliente decidió escribir, no un documento con validez. El código de verificación permite cruzar contra la fila de Validaciones, pero el punto de control final es el dueño revisando el total antes de despachar. Está documentado como paso obligatorio en el manual del dueño.

8 Capacidad y límites

Números medidos, no estimados. Se miden con `limites.js`, que siembra catálogos de distinto tamaño en la hoja y cronometra el backend real y la tienda en un navegador.

Tamaño del catálogo

PRODUCTOS	RESPUESTA DE <code>?A=CATALOGO</code>	TIENDA LISTA	TARJETAS EN EL DOM
50	11 KB	0,36 s	25
150	30 KB	0,32 s	25
300	59 KB	0,28 s	25
600	117 KB	0,30 s	25
1.000	195 KB	0,32 s	25

El tiempo de pintado no crece porque la paginación mantiene el DOM en 25, 50 o 100 tarjetas pase lo que pase. **Lo que crece es el JSON**, y ese viaja por datos móviles. Los números de arriba son con descripciones cortas; con descripciones reales de ~200 caracteres y URLs de fotos, cuenta el doble.

Recomendación: hasta 300 productos cómodo, 500 el tope práctico. Más allá, el catálogo pesa más que la propia página y conviene otro diseño (carga por categoría, o un backend de verdad).

Concurrencia

El techo es la cuota de Google: **30 ejecuciones simultáneas** de Apps Script. Pero no toda visita consume una:

- **Ver la tienda** casi nunca: el catálogo va en caché 60 s, así que una sola ejecución sirve a todos los que entren en ese minuto.
- **Aplicar un cupón o enviar un pedido** sí consume una, y dura uno o dos segundos.

En la práctica: **~25 personas validando en el mismo instante**. La 26 recibe un fallo, la tienda reintentará dos veces sola, y si no lo logra el pedido sale marcado como no verificado en vez de perderse.

Tráfico y almacenamiento

RECURSO	TOPE DEL PLAN GRATUITO	QUÉ SIGNIFICA AQUÍ
Cloudinary (fotos)	25 créditos/mes ≈ 25 GB de entrega	Es el primer techo. Una visita nueva que ve una página de 25 productos descarga ~1 MB de fotos: ~25.000 visitas nuevas al mes . Las repetidas casi no cuentan, porque el navegador las guarda.
Netlify (la página)	Plan gratuito por créditos, 300/mes	La página pesa ~100 KB. No es el cuello de botella. Al agotarse, el sitio queda pausado hasta el mes siguiente.
Google Sheets	10 millones de celdas	Muy por encima del tope que impone el propio script.
Filas de Pedidos	20.000 (MAX_FILAS)	Una fila por línea de pedido: 6.000 a 10.000 pedidos . Al llegar, el script se niega a escribir con un mensaje claro en vez de corromper la hoja: hay que archivar y vaciar.
Correo	100 al día	El resumen gasta 1.
Disparadores	90 minutos al día	El recálculo horario tarda segundos. Sobra.
Ejecución	6 minutos cada una	La más lenta —recalcular tablero con miles de filas— va muy por debajo.

LOS PLANES GRATUITOS CAMBIAN

Los topes de Netlify y Cloudinary de esta tabla son de **septiembre de 2026**, y ambos proveedores los han cambiado antes. Verifícalos al montar cada tienda. Las cuotas de Google Apps Script son las más estables de las tres.

Rendimiento del cliente

- Catálogo cacheado 60 s con `CacheService` ; se invalida al confirmar un pedido o al cambiar un color.
- Fotos por Cloudinary con `f_auto,q_auto,c_fill,ar_1:1` y ancho según el uso: 600 px tarjeta, 900 px galería, 160 px miniatura.
- Espera de 15 s por petición y 2 reintentos silenciosos, por la red móvil.
- Paginación de 25 / 50 / 100.

Lo que el sistema no hace, por diseño

- Sin pasarela de pagos ni facturación electrónica.
- Sin generación de guías ni integración logística.
- Sin usuarios ni permisos: quien tenga la hoja lo puede todo.
- Sin datos personales almacenados: no hay CRM ni historial por cliente.
- Sin picos: una campaña masiva concentrada en una hora excede la concurrencia.

Cada una de esas ausencias es lo que sostiene el costo en \$0/mes. El punto de quiebre razonable son **~30 pedidos al día** o la necesidad de cobrar en línea: ahí ya hay volumen para pagar una plataforma con mensualidad.

9 Despliegue

Backend

- 1 **Hoja nueva**
en Google Sheets. *Extensiones* → *Apps Script*.
- 2 **Pegar** `apps-script.gs` **completo**
, reemplazando todo.
- 3 **Ejecutar** `instalar()`
. Está de primera en el archivo a propósito: el desplegable del editor toma la primera función, y ejecutar otra por accidente es un clásico. Crea las nueve pestañas, siembra datos de ejemplo y programa los disparadores.
- 4 **Implementar** → **Nueva implementación** → **Aplicación web**.
Ejecutar como: yo. Acceso: cualquier usuario. Copiar la URL que termina en `/exec` .

5

Pegar esa URL

en `SCRIPT_URL` dentro de `index.html` . Es lo único que se escribe a mano en todo el archivo.

Frontend

Arrastrar la carpeta `publicar/` sobre la pestaña *Deploys* del sitio en Netlify. Sin CI, sin build.

Cuando cambia el título, la descripción o el dominio

El `<title>` , las etiquetas Open Graph, el canonical y el favicon tienen que ser HTML estático: los rastreadores no ejecutan JavaScript. Menú **Orgánico** → **Generar configuración para index.html** abre una ventana con los dos bloques listos para copiar. El primero reemplaza lo que hay entre las marcas `CONFIGURACIÓN DE ESTA TIENDA` y `FIN DE LA CONFIGURACIÓN` ; el segundo, las cuatro constantes del `<script>` . La URL `/exec` se pega a mano: es lo único que el script no puede saber de sí mismo. Después, republicar.

Los dos bloques generados

Nada de `index.html` se edita a mano salvo `SCRIPT_URL` . Todo lo demás que alguna vez hubo que tocar ahora lo escribe el Apps Script y se pega entre marcas:

MENÚ	QUÉ GENERA	CUÁNDO
Generar configuración para index.html	El bloque del <code><head></code> (title, Open Graph, canonical, favicon, theme-color) y las cuatro constantes del <code><script></code> .	Cambió el título, la descripción, el dominio o los colores de marca.
Generar inventario para index.html	<code>const ENVIOS</code> y <code>const PRODUCTOS</code> : el catálogo de respaldo, con precios y stock del momento, en JavaScript válido.	Cada tanto, para que el paracaídas esté al día.

La lista de cupones desapareció del archivo: los cupones solo existen en la hoja y solo la hoja los valida. Un paso manual menos en el despliegue, y una superficie menos.

La hoja como interfaz

`presentarHojas()` corre al instalar y desde el menú. Deja encabezados con el color de la marca, filas y columnas fijas, anchos, formatos de moneda y fecha, y —lo importante— **listas desplegables** en todo campo donde una errata rompe el sistema:

Estado del pedido, Destacado, Activo, Tipo de cupón, correo_siempre.
Categoría sugiere pero deja escribir, porque el negocio crece.

Los tres colores de marca se pueden **pintar** con el relleno nativo de Sheets:

`sincronizarColores()` lee el fondo con `getBackground()` y escribe el hexadecimal. El camino inverso lo hace `alEditar`, que pinta la celda con el valor recién escrito.Cuál gana se decide por cuál cambió, y por eso las dos direcciones no se pelean.

10 Replicar para otro cliente

La tienda es genérica. Cambiando solo la pestaña `Configuración`, el mismo `index.html` se convierte en otro negocio —nombre, colores, textos, WhatsApp, legales y favicon incluidos—. Hay una prueba automática que lo verifica convirtiendo la tomatara en una panadería.

1 Hoja y script

— los cinco pasos de despliegue del backend. *~15 min*

2 Configuración

— llenar la pestaña con los datos del cliente: nombre, WhatsApp, textos de portada, dos colores de marca, datos de la empresa para los legales. *~10 min*

3 Catálogo

— una fila por producto. Las fotos a Cloudinary, las URL en la columna Imágenes separadas por `|`. *~15 min, según cuántos productos*

4 Envíos y cupones

— zonas con sus valores; cupones si los hay. *~5 min*

5 Bloque estático

— menú → Generar configuración → pegar los dos bloques en `index.html` → pegar la URL `/exec`. *~5 min*

6 Publicar

— arrastrar `publicar/` a Netlify. *~2 min*

7 Cerrar el circuito

— respuesta automática de WhatsApp Business con los datos de pago y la advertencia; correo del dueño en `correo_resumen`. *~5 min*

LO QUE SÍ HAY QUE REVISAR CADA VEZ

La imagen `compartir.jpg` (1200×630) es del negocio y hay que reemplazarla. Los datos legales `empresa_*` son obligatorios en Colombia por la Ley 1480: si quedan entre corchetes, la frase no se muestra, pero el vendedor queda sin identificar. Y la CSP de `_headers` lista los dominios de imágenes permitidos: si el cliente usa otro proveedor de fotos, hay que agregarlo ahí **y** en la etiqueta `meta` .

11 Pruebas

525 comprobaciones en 16 baterías, sobre el código real. Corren en Node con Playwright y Chromium.

La pieza clave es `gas.js` : un emulador de Google Apps Script y Sheets que **carga** `apps-script.gs` **tal cual** y le inyecta el entorno de Google (`SpreadsheetApp` , `CacheService` , `LockService` , `MailApp` , `HtmlService` ...). Existe porque una versión anterior del banco de pruebas reimplementaba el backend a mano: validaba la imitación, no el código del cliente. El servidor de pruebas sirve `index.html` reescribiendo `SCRIPT_URL` , de modo que el navegador habla con el backend real.

BATERÍA	QUÉ CUBRE
e2e · test · movil	Compra completa, formulario, red móvil que se cae, reintentos.
pedidos	Registro, deduplicación, inventario, anulación, inyección de fórmulas.
val · cat · hoja	Validación de cupones y envíos, catálogo desde la hoja, cambios en caliente.
config	Convierte la tienda en otro negocio cambiando solo la hoja.
tablero	Cada número del tablero, y que el diseño esté puesto.
correo	Destinatarios, hora, una vez al día, contenido, adjunto mensual, fallos.
enlace	Enlace por producto, compartir, botón atrás, producto inexistente.
presentacion	Formato de todas las hojas, listas desplegables, colores pintables y el catálogo de respaldo generado.
version · menu · pag · sec2	Desfase de versión, menú y ventana de configuración, paginación, seguridad.

12 Decisiones y límites

Decisiones deliberadas

- **La hoja es la única fuente de verdad** de precios, stock, envíos, cupones, marca y textos. El archivo solo guarda un respaldo que evita que la tienda se caiga si Google no responde.
- **Fallo cerrado en cupones:** si la hoja no responde, no se aplica descuento. Un cupón aplicado sin validar es plata perdida.
- **Fallo abierto en catálogo:** si la hoja no responde, la tienda sirve el catálogo del archivo. Una tienda vacía es peor que una desactualizada.
- **Los gráficos del tablero se dibujan con bloques (■) y no con SPARKLINE:** las fórmulas de Sheets cambian de separador según el idioma de la hoja, y una fórmula escrita desde el script se rompe con solo cambiar el idioma. Un bloque de texto se ve igual en todas partes.
- **El correo se revisa cada hora** en vez de programar un disparador a una hora fija, porque la hora vive en la hoja. Se cura solo si Google se salta una ejecución.

Límites conocidos

- **La vista previa de un enlace de producto** es la de la tienda, no la del producto. Los rastreadores no ejecutan JavaScript. Arreglarlo pediría una página por producto y un paso de build.
- **El total del mensaje es editable** por el cliente antes de enviarlo. Mitigado con el código de verificación y el paso de revisión del dueño.
- **El contador de usos de cupón** se actualiza cada hora: un cupón de un solo uso conviene apagarlo a mano.
- **Sin pasarela de pagos.** El cobro se acuerda por chat. Es parte de la premisa de costo cero, no una omisión.
- **Techo de concurrencia:** 30 ejecuciones simultáneas de Apps Script.

13 Mantenimiento

Diagnóstico

Menú **Orgánico** → **Diagnóstico** imprime en el Registro de ejecución: la versión del código, la hoja a la que está unido, qué pestañas faltan y cuántas filas tiene cada una. Es el primer paso ante cualquier síntoma.

SÍNTOMA	CAUSA HABITUAL
La tienda no muestra productos	Pestaña renombrada, o <code>instalar()</code> nunca se ejecutó.
Los pedidos no se graban	Implementación desactualizada. Publicar versión nueva.
Aviso de versión en consola	Lo mismo. Es exactamente lo que ese aviso detecta.
Las fotos no cargan	Dominio de imágenes no listado en la CSP, en <code>meta</code> o en <code>_headers</code> .
Filas en la pestaña Errores	Payload que el backend no entendió. Trae los primeros 200 caracteres recibidos.
El correo no llega	Sin destinatario, no era la hora, o no había nada que contar. Probar con el menú.

Rutina

- La hoja crece una fila por línea de pedido. A las 20.000 filas conviene archivar el año anterior en una copia y vaciar; el script avisa antes de llegar al tope.
- El catálogo de respaldo dentro de `index.html` conviene refrescarlo de vez en cuando: solo se usa si Google no responde, pero es lo que ve el cliente ese día.
- Los disparadores se recrean al ejecutar `instalar()`; ejecutarlo de nuevo no duplica nada.